

R4.11 Conception et développement d'applications mobiles



Jean-François Remm

IUT Nantes département Informatique

2022–2023

Extrait du PN

- Problématiques de la mobilité (par ex. : autonomie, robustesse)
- Programmation pour un système mobile
- Connectivité, utilisation de "Services Web" (*Web services*)

Organisation du module

Volume horaire : $\simeq 16H$

- par semaine : 1H20 CM + 4H TD

Modalité d'évaluation :

- note de miniprojet $\frac{1}{4}$
- note de test $\frac{3}{4}$

Équipe pédagogique

- CM : Jean-François Remm
- TD : Olivier Boutin, Arnaud Lanoix et Jean-François Remm

Développement Mobile

- application mobile = application développée pour :
 - smartphone,
 - tablette,
 - montres connectées (*Wear*),
 - véhicules,
 - ...
- écosystèmes complets (OS, “market”, SDK, ...) :
 - 1 android
 - 2 ios
 - 3 ...
- développement mobile :
 - 1 application web : html5 + css + js
 - 2 application hybride : Ionic
 - 3 application cross-platform : Flutter, Apache Cordova, ...
 - 4 développement natif

- 1 Plateforme Android
- 2 Outils de développement
- 3 Application Android
- 4 Activité

Plateforme Android

- pile logicielle complète : SE → applications de base
- dédiée aux terminaux mobiles :
 - smartphones,
 - tablettes,
 - montres connectées, ...
- sdk pour la création d'applications :
 - outils de développement
 - bibliothèques
 - documentation
- se code en **java** ou **Kotlin**
- site de référence :
 - <http://developer.android.com>

Historique

- octobre 2003 : conception d'un OS mobile par Android Inc. (co-fondé par Andy Rubin)
- août 2005 : rachat d'Android Inc. par Google
- novembre 2007 : création du consortium Open Handset Alliance (Google + industriels) et de l'Android Open Source Project (AOSP), version beta sous licence Open Source Apache
- octobre 2008 : première version avec téléphone : HTC G1/Dream
- 2014 : version "importante" : 5.X Lollipop
- actuellement sortie annuelle d'une version majeure
- 2022 : dernière version en date 13

Versions

Les différentes versions d'Android :

- portaient des noms de sucreries (en anglais)
- suivent une logique alphabétique (de A vers Z)
- possèdent un numéro d'API (*API level*) qui s'incrémente

Version	Nom	Sortie	API
4.0	Ice Cream Sandwich	10/2011	14-15
4.1 - 4.3	Jelly Bean	07/2012-07/2013	16-18
4.4	KitKat	10/2013	19-20
5.0 - 5.1	Lollipop	11/2014-03/2015	21-22
6.0	Marshmallow	10/2015	23
7.0 - 7.1	Nougat	08/2016-10/2016	24-25
8.0 - 8.1	Oreo	08/2017-12/2017	26-27
9.0	Pie	2018	28
10.0	Q	2019	29
11.0	R	2020	30
12.0	S	2021	31-32
13.0	Tiramisu	2022	33

Architecture Android



Figure – "Android-System-Architecture" (source : Wikimedia Commons)

Noyau Linux



standard :

- gestion de la sécurité
- gestion des processus et de la mémoire
- E/S fichier et réseau
- gestion de périphériques

modifié pour android :

- gestion alimentation
- IPC
- gestion mémoire (*Kernel Memory Killer*)
- ...

Bibliothèques système



- écrites en C et C++
- libc → syscall : création process, allocation mémoire
- surface manager : rendu 2D/3D
- media framework : lecture/enregistrement audio/vidéo
- webkit : moteur de rendu de pages web
- openGL : moteur graphique
- sqlite : SGBDR

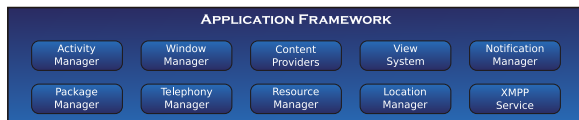
Android Runtime



deux composants principaux :

- ❶ bibliothèque java core :
 - classes de base (`java.*`, `javax.*`) : structures de données, IO, réseau, XML...
 - classes spécifiques android `android.*`
 - services internet/web `org.*` (SAX, DOM, JSON)
 - junit
- ❷ machine virtuelle (Dalvik puis ART)

Gestionnaires



- *Activity Manager* : gestion du cycle de vie d'une application
- *Window Manager* : gestion des fenêtres composant une application
- *ContentProvider* : partage de données inter-applications
- *Views System* : composants graphiques (boutons, icônes, listes, ...) pour UI
- *Notification Manager* : permet de placer des informations dans la barre de notification
- *Package Manager* : gestion des applications installées
- *Telephony Manager* : gestion de la téléphonie
- *Ressource Manager* : gestion des ressources (Strings, graphiques, mise en page)
- *Location Manager* : information sur mouvement et position
- *XMPP Manager* : *eXtensible Messaging and Presence Protocol* → messagerie instantanée

Applications de base



- écran d'accueil
- contact
- téléphone
- navigateur web
- client mail
- ...

- 1 Plateforme Android
- 2 Outils de développement
- 3 Application Android
- 4 Activité

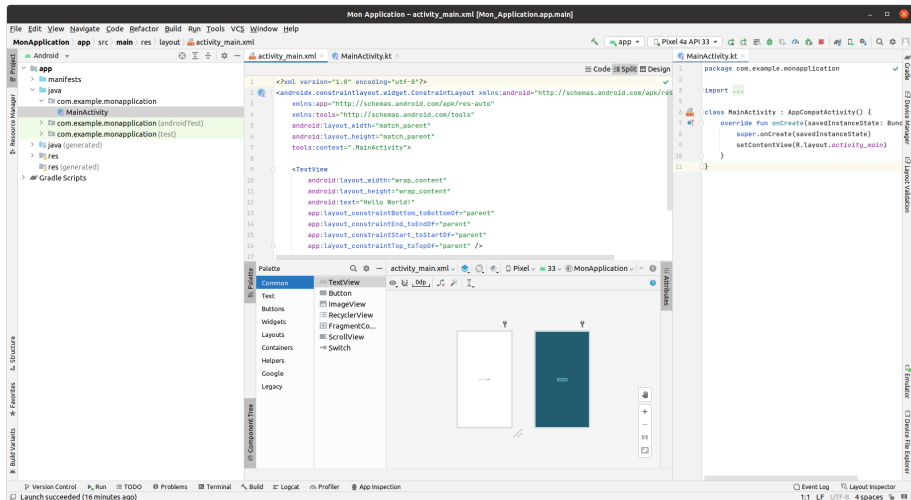
Android SDK (*Software Development Kit*)

ensemble complet d'outils de développement :

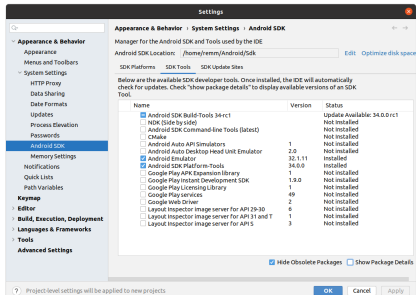
- Platform : `android.jar`
 - `.class` : sous ensemble java de base + `android.` + `org.`
 - `ressources`
- Tools :
 - `sdkmanager` : → Android SDK Manager
 - `avdmanager` : → Android Virtual Device Manager
- `emulator` : émulateur basé sur QEMU
- Platform-tools → `adb`, `sqlite3`
- Build-tools → `aapt`, `dx`, ...

Android Studio

- environnement de développement officiel
- la plupart des outils, par le passé autonomes, y sont intégrés

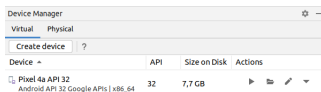


- versions de l'api android installés
- sources des classes android
- images système des émulateurs :
system-images (avec ou sans apis google)
- outils
- ...

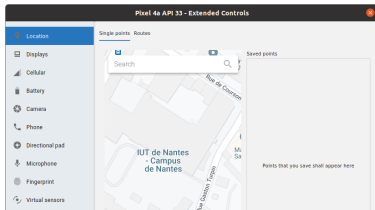
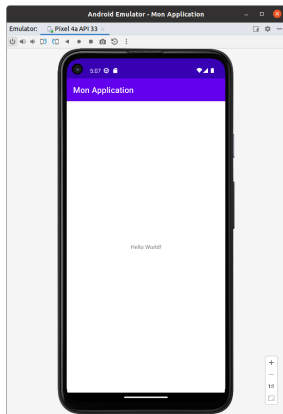


Android Device Manager

- permet de créer des configurations à émuler
- intègre des définitions de terminaux existants
- *Virtual Device* = image système (iso) + configuration matérielle (écran + proc + mem + ...)



Emulator



Dalvik / ART (Android Runtime)

- exécution de bytecode java
- n'interprète pas le bytecode java mais des `.dex` (*Dalvik EXecutable*)
- JVM classique : à pile
- Dalvik : basé registre

java/kotlin ----> `.class` ----> `.dex` + ressources
 compilation dx

- Dalvik remplacé depuis Lollipop par ART (*Android RunTime*)
- type de compilation :
 - Dalvik : compilation JIT (Just In Time)
 - ART : compilation (AOT ahead-of-time)
- ART : compilation à l'installation ⇒ meilleures performances, économie de batterie, surtout en stockage (30%)

source :

[https://fr.wikipedia.org/wiki/Dalvik_\(machine_virtuelle\)](https://fr.wikipedia.org/wiki/Dalvik_(machine_virtuelle))

[https://fr.wikipedia.org/wiki/ART_\(Android\)](https://fr.wikipedia.org/wiki/ART_(Android))

<https://source.android.com/docs/core/runtime>

ADB (Android Debug Bridge)

- à l'origine, utilitaire en ligne de commande
- maintenant, plutôt via Android Studio
- fonctionnement client/serveur avec un émulateur lancé
- utilisation directe compliquée mais toujours possible
- exemples :

- lister les périphériques :

```
adb devices
List of devices attached
0698061e0036d349 device
emulator-5554      device
```

- copie(push)/lecture(pull) de fichier :

```
adb push monfichier.mp3 /sdcard/Music
```

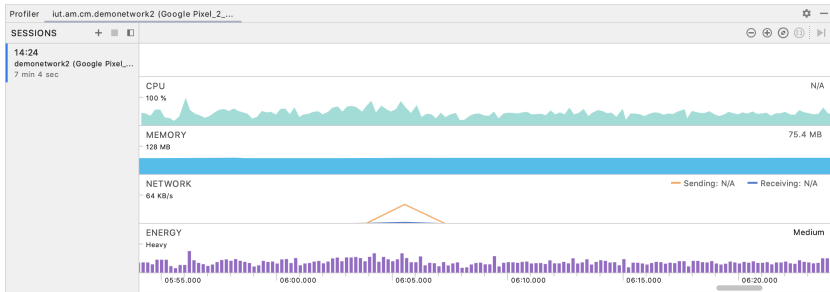
- (de)installation (install/uninstall) d'une application :

```
adb install monApp.apk
```

- shell direct possible

```
adb shell
$ sqlite3 /data/data/com.example.app/databases/rssitems.db
```

Android Profiler



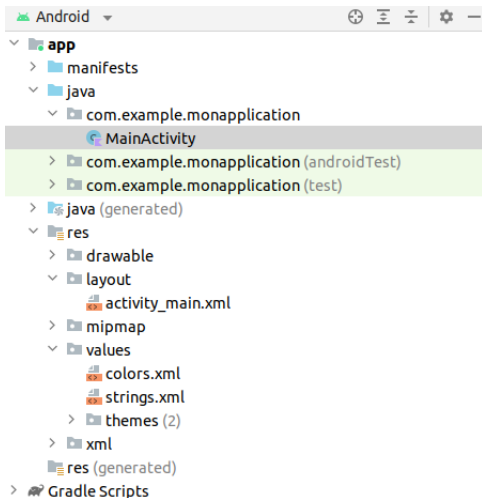
- 1 Plateforme Android
- 2 Outils de développement
- 3 Application Android**
- 4 Activité

Application Android

application android :

- écrite en Java ou Kotlin
- déployée sous forme d'un fichier apk (*Android Package*)
- chaque application s'exécute sous l'identité d'un user différent sur le système linux sous jacent
- constituants :
 - 1 fichiers XML :
 - ★ `AndroidManifest.xml` : description de application (version de l'API utilisée, activité principale, etc)
 - ★ `*.xml` : description des ressources (texte, image, "layout", etc)
 - 2 code kotlin/java : spécification de la partie dynamique de l'application
 - 3 fichiers de ressources (image, son, ...)

Structure d'une application



AndroidManifest.xml (simplifié)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="dev.mobile.monapplication">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.MonApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Ressources

- images, fichiers de configuration, description des interfaces graphiques
- séparées du code source
- placées dans un répertoire `/res` de l'application
- incorporées au binaire de l'application
 - `drawables` : images (i.e. fichiers png, gif, ...) ou des fichiers XML.
 - `layout` : définitions en XML des interfaces graphiques des écrans ou fragments d'écran
 - `menu` : fichiers XML décrivant les menus de l'application
 - `font` : fichiers XML ou (par exemple) `.ttf`
 - `values` : définitions de chaînes, de couleurs, de dimensions, ... en XML (fichier général `values.xml` ou spécialisés `strings.xml`, `colors.xml`, `styles.xml`, ...)

source : <https://developer.android.com/guide/topics/resources>

Ressources : exemple

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="nom">Démonstration</string>
    <string name="hello_world">Hello world!</string>

    <string-array name="liste">
        <item>IOS</item>
        <item>Android</item>
        <item>Windows Phone</item>
    </string-array>

    <color name="bleu">#ff0000ff</color>

    <dimen name="unemarge">10dp</dimen>

</resources>
```

Ressources alternatives

- une application peut proposer des ressources alternatives
- par la création d'un répertoire dans `res/`.
- exemple :

```
res/  
  drawable/  
    icon.png  
    background.png  
  drawable-hdpi/  
    icon.png  
    background.png
```

- en fonction de $\neq$ critères :
 - tailles d'écran : `small`, `normal`, `large` et `extra-large`.
 - densité d'écran : `ldpi`, `mdpi`, `hdpi`, `xdpi`, `xxdpi`, ... en fonction de la capacité en dpi (*dots per inch*) de l'écran
 - langue : `en`, `fr`
 - orientation de l'écran `port`, `land`
 - pays et/ou opérateur : MCC (*mobile country code*) et MNC (*mobile network code*). ex : `mcc208-mnc00` France-orange
 - ...

Classe R

- R classe générée par aapt permet d'accéder aux ressources
- une classe interne par type de ressource (ex : R.string)
- une constante entière par ressource (ex : R.string.hello.)
- cette constante est un ID qui permettra de retrouver la ressource

```
public final class R {  
    public static final class id {  
        public static final int action_settings=0x7f080000;  
    }  
    public static final class layout {  
        public static final int activity_main=0x7f030000;  
    }  
    public static final class string {  
        public static final int app_name=0x7f060000;  
        public static final int deux=0x7f060001;  
        public static final int trois=0x7f060002;  
        public static final int un=0x7f060003;  
    }  
}
```


Accès aux ressources : XML

- par exemple pour utiliser une string définie dans un fichier de ressources dans une interface graphique
- syntaxe :
@ [<package_name>:]<resource_type>/<resource_name>
- exemple :

```
/res/values/strings.xml
```

```
<resources>  
    <string name="hello_world">Hello world!</string>  
</resources>
```

```
/res/layout/ig.xml
```

```
<TextView android:text="@string/hello_world"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content" />
```

Accès aux ressources : kotlin

- syntaxe :

```
[<package_name>.]R.<resource_type>.<resource_name>
```

- méthode de base :

```
fun <T : View!> findViewById(id: Int): T
```

- exemple :

```
// chargement d'une UI pour l'activité  
setContentView(R.layout.main_screen);
```

```
// positionne le texte d'un label
```

```
val msgTextView = findViewById<TextView>(R.id.msg)
```

```
msgTextView.text(R.string.hello_message)
```

Style et thème

- style = attributs définissant l'aspect visuel d'un composant (ex : un bouton)
- style, comme en css = dimensions, marges internes et externes, couleurs, polices, ...
- style défini dans des fichiers XML de ressources

```
<resources>
  <style name="CodeFont">
    <item name="android:textColor">#00FF00</item>
    <item name="android:typeface">monospace</item>
    <item name="android:textSize">16dp</item>
  </style>
</ressources>
```

- utilisable dans un layout

```
<TextView
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text="Hello World!"
  android:textAppearance="@style/CodeFont"
/>
```

Style et thème

- pas de cascade comme en css, mais possibilité d'héritage

```
<style name="BlueFont" parent="@style/CodeFont">  
  <item name="android:textColor">#0000FF</item>  
</style>
```

- qui peut être implicite

```
<style name="CodeFont.Red">  
  <item name="android:textColor">#FF0000</item>  
</style>
```

- application d'un style

```
<TextView  
  style="@style/CodeFont"  
  android:text="@string/hello" />
```

- thème = style de toute une application ou activité
- application d'un thème

```
<application android:theme="@style/CustomTheme">  
<activity android:theme="@android:style/Theme.Dialog">
```

source :

<https://developer.android.com/guide/topics/ui/look-and-feel/themes.html>

Style et thème

- exemple :

```
values/colors_material.xml
```

```
<style name="Theme.Material">
```

```
...
```

```
<item name="colorBackground">@color/background_material_dark</item>
```

```
<item name="textAppearance">@style/TextAppearance.Material</item>
```

```
values/colors_material.xml
```

```
<color name="background_material_dark">@color/material_grey_850</color>
```

```
<color name="material_grey_850">#ff303030</color>
```

```
values/styles_material.xml
```

```
<style name="TextAppearance.Material">
```

```
<item name="textColor">?attr/textColorPrimary</item>
```

```
<item name="textColorHint">?attr/textColorHint</item>
```

```
<item name="textColorHighlight">?attr/textColorHighlight</item>
```

```
<item name="textColorLink">?attr/textColorLink</item>
```

```
<item name="textSize">@dimen/text_size_body_1_material</item>
```

```
<item name="fontFamily">@string/font_family_body_1_material</item>
```

```
</style>
```

Accès aux ressources de style

- par exemple pour utiliser une valeur définie dans le thème courant
- syntaxe :
? [<package_name>:] [<resource_type>/] <resource_name>
- exemple :

```
<EditText id="text"  
    android:layout_width="fill_parent"  
    android:layout_height="wrap_content"  
    android:textColor="?android:textColorSecondary"  
    android:text="@string/hello_world" />
```

Accès aux ressources standard

- utilisation de `android.R`.
- exemple :

```
ArrayAdapter<String> (this,  
                      android.R.layout.simple_list_item_1, tab)
```

```
platforms/android-NN/data/res/layout/simple_list_item_1.xml
```

```
<TextView xmlns:android="http://schemas.android.com/apk/res/android"  
    android:id="@android:id/text1"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:textAppearance="?android:attr/textAppearanceListItemSmall"  
    android:gravity="center_vertical"  
    android:paddingStart="?android:attr/listPreferredItemPaddingStart"  
    android:paddingEnd="?android:attr/listPreferredItemPaddingEnd"  
    android:minHeight="?android:attr/listPreferredItemHeightSmall" />
```

Composants d'une application

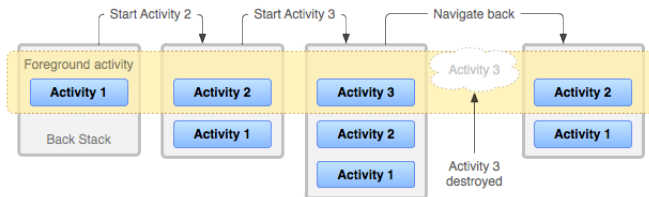
- ❶ *Activity* : un écran doté d'une interface graphique
- ❷ *Service* : exécutés en arrière plan, permettent des opérations longue ou une interaction avec des processus distant
- ❸ *BroadcastReceiver* : écoute et réaction à des événements (`Intent`)
- ❹ *Content Provider* : stockage et partage de données entre applications

- 1 Plateforme Android
- 2 Outils de développement
- 3 Application Android
- 4 Activité**

- application Android = des écrans
- un écran = une Activité (`Activity`)
- exemples :
 - visualiser un mail
 - afficher une fenêtre de login
 - ...
- peut comporter plusieurs fragments (`Fragment`)
- un Activité peut en lancer une autre
- les Activités peuvent être interrompues, reprises, tuées, ...

BackStack

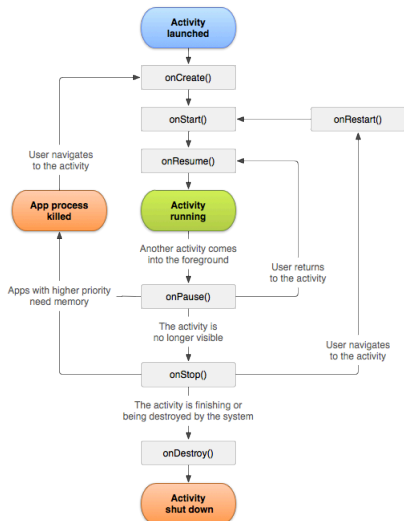
- les activités sont gérées à l'aide d'une pile
- une activité peut lancer une autre activité de la même application (ex : liste de mails → détail)
- une activité peut lancer une autre activité d'une autre application (ex : appareils photo → composeur de mms).
- back, home, plus de ressources ?



source :

<https://developer.android.com/guide/components/activities/tasks-and-back-stack.html>

Cycle de vie d'une activité



- événements :

- action de l'utilisateur : `back`, `home`
- événements systèmes

- visibilité :

- visible et au premier plan : `onResume()` - `onPause()`
- visible : `onStart()` - `onStop()`

<https://developer.android.com/guide/components/activities/activity-lifecycle.html>

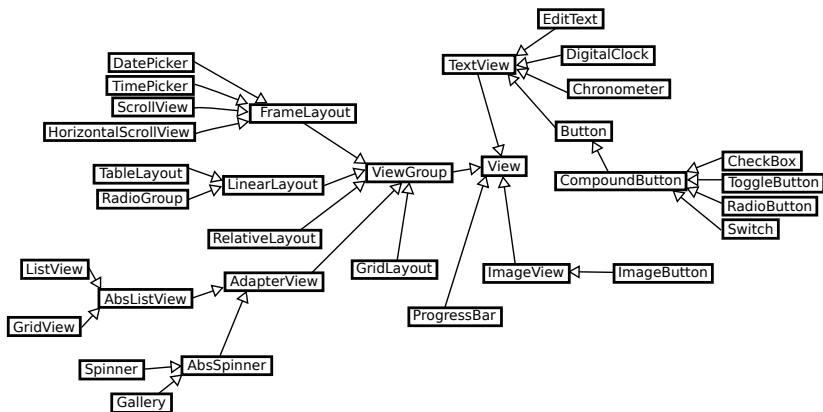
Interface graphique

- une arborescence de composants (`View`) et `ViewGroup`
- positionnés à l'aide de gestionnaire de placement (*layout*)
- avec de la gestion d'événements

Vue d'ensemble

- `android.view.View` classe mère de tous les composants graphiques
- occupe une zone rectangulaire et gère affichage et gestion des événements de cette zone
- nombreuses sous-classes dont `ViewGroup` qui est la classe mère des conteneurs
- gestion propriétés : icône à afficher, texte par défaut, label, ..., focus, visibilité et événements.

Vue d'ensemble



- deux solutions pour définir l'interface graphique :
 - 1 en code java/kotlin pur
 - 2 par un fichier XML
 - 3 en code kotlin "Compose"

Layout : Kotlin

```
class MonActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        val rl = RelativeLayout(this)  
        val rlp = RelativeLayout.LayoutParams(  
            RelativeLayout.LayoutParams.MATCH_PARENT,  
            RelativeLayout.LayoutParams.MATCH_PARENT  
        )  
        val tv = TextView(this)  
        tv.text = "Test"  
  
        val lp = RelativeLayout.LayoutParams(  
            RelativeLayout.LayoutParams.WRAP_CONTENT,  
            RelativeLayout.LayoutParams.WRAP_CONTENT  
        )  
        lp.addRule(RelativeLayout.CENTER_IN_PARENT)  
  
        tv.layoutParams = lp  
        rl.addView(tv)  
        setContentView(rl, rlp)  
    }  
}
```

Layout : XML

```
<?xml version="1.0" encoding="utf-8"?>
<ViewGroup
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+[package:]id/resource_name"
    android:layout_height=["dimension" | "match_parent" | "wrap_content"]
    android:layout_width=["dimension" | "match_parent" | "wrap_content"]
    [ViewGroup-specific attributes] >
    <View
        android:id="@+[package:]id/resource_name"
        android:layout_height=["dimension" | "match_parent" | "wrap_content"]
        android:layout_width=["dimension" | "match_parent" | "wrap_content"]
        [View-specific attributes] >
        <requestFocus/>
    </View>
</ViewGroup >
    <View />
</ViewGroup>
    <include layout="@layout/layout_resource"/>
</ViewGroup>
```

Layout : XML

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Test" />

</RelativeLayout>
```

Layout : Kotlin Compose

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            CmComposeBasicTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colors.background
                ) {
                    TexteMilieu("Test")
                }
            }
        }
    }
}

@Composable
fun TexteMilieu(name: String) {
    Column(horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center) {
        Text(modifier = Modifier.padding(16.dp),
            text = "$name!")
    }
}
```

Propriétés : XML vs Java/Kotlin ... sur un exemple

- XML : `android:visibility` **valeur possibles** : `visible`, `invisible` et `gone`
- Java : `public void setVisibility (int visibility)`
- Kotlin : `open fun setVisibility(visibility: Int): Unit`
valeurs possibles : `VISIBLE`, `INVISIBLE`, et `GONE`.

Propriétés des composants

- **visibilité** : `android:visibility` **ou** `visibility`
- **fond** : `android:background` **ou** `background`
- **padding** : `android:padding` **ou**
`setPadding (int start, int top, int end, int bottom)`
- **gestion du focus** : `android:nextFocusForward`
`nextFocusForwardId`
- ...

En particulier :

<http://developer.android.com/reference/android/view/View.html>

Propriété ID

- identifiant pour chaque composant graphique
- assigné dans le layout XML :

```
<Button  
    android:id="@+id/bouton1"  
    ...  
    android:text="Bouton 1"/>
```

- utilisé pour retrouver la référence d'un composant en XML

```
<Button  
    android:layout_below="@id/bouton1"  
    ...  
    android:text="Bouton 2"/>
```

- utilisé pour retrouver la référence d'un composant en code

```
val bouton1 : Button = findViewById<Button>(R.id.bouton1)
```

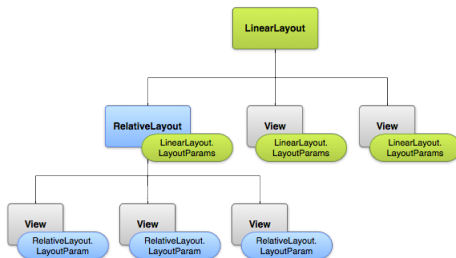
Layout XML / association avec l'activité

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```


- héritent de `ViewGroup`
- gèrent le placement des éléments enfants (inclus)
- exemple :
 - `ConstraintLayout` : définition de contrainte des composants les uns par rapport aux autres ou au parent
 - `RelativeLayout` : positionnement relatif des composants les uns par rapport aux autres ou au parent
 - `LinearLayout` : alignement en ligne ou colonne
 - `FrameLayout` : un élément
 - `GridLayout` : grille de composant placés (x,y)
 - ...

Paramètres : LayoutParams

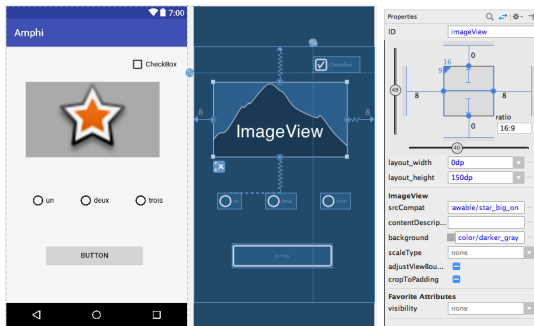


- chaque layout définit des paramètres
- des attributs `layout_xxx` définis dans `ViewGroup.LayoutParams` ou l'une de ses sous-classes
- valables pour les composants inclus dans le layout

Paramètres généraux

- `ViewGroup.LayoutParams` définit :
 - ① `android:layout_height` taille du composant en hauteur
 - ② `android:layout_width` taille du composant en largeur
- valeurs possibles :
 - ① `match_parent` : dimensionné à la taille que permet le parent
 - ② `wrap_content` : dimensionné à la taille de son contenu
 - ③ une taille
 - ④ une taille à 0dp qui correspond à une valeur `match_constraints` pour `ConstraintLayout`
- les autres paramètres dépendent du layout

ConstraintLayout



- placement des composants par définition de contraintes :
 - positions relatives
 - marges et biais
 - dimensions
 - ...
- sous la forme `layout_constraint<Source>_to<Target>Of="idComposant"`
 - `<Source>/<Target> = Left/Right/Bottom/Top`

Exemple ConstraintLayout

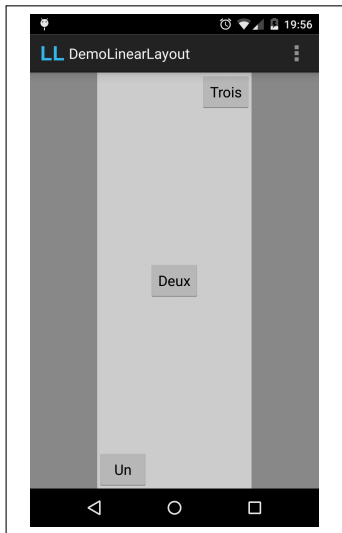
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_marginTop="96dp"
        android:text="Hello World!"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/button" />
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginTop="120dp" android:layout_marginEnd="8dp"
        android:text="Button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.677"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

LinearLayout



- organisation des composants en une ligne horizontale ou verticale
- propriétés :
 - `android:orientation` : **sens** horizontal **ou** vertical
 - `android:gravity` : **attraction du contenu** top, bottom, fill, center_vertical, ...
- pour les composants : `LinearLayout.LayoutParams`
 - `android:layout_weight` : **répartition de l'espace supplémentaire**
 - `android:layout_gravity` : **attraction du composant dans son layout**

Exemple LinearLayout



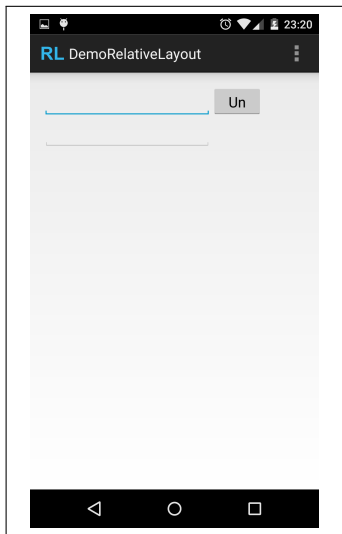
```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center_horizontal" >
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_weight="1" >
        <Button
            ...
            android:layout_gravity="bottom"
            android:text="Un" />
        <Button
            ...
            android:layout_gravity="center"
            android:text="Deux" />
        <Button
            ...
            android:layout_gravity="top"
            android:text="Trois" />
    </LinearLayout>
</LinearLayout>
```

RelativeLayout



- organisation des composants les uns par rapport aux autres
- propriétés :
 - `android:gravity` : attraction du contenu `top`, `bottom`, `fill`, `center_vertical`, ...
- pour les composants : `RelativeLayout.LayoutParams`
 - `layout_below`, `layout_above` : placement relatif à d'autres
 - `alignRight`, `alignTop`, ... : alignement relatif à d'autres
 - `alignParentRight`, `alignParentTop`, ... : alignement relatif au parent

Exemple RelativeLayout

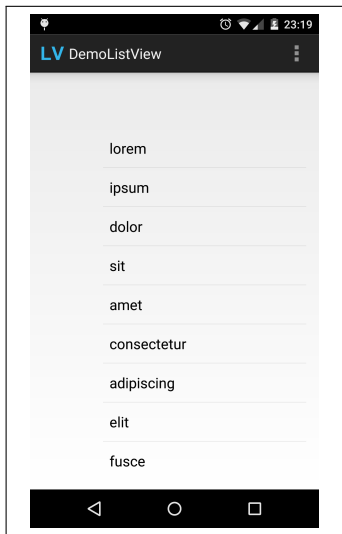


```
<RelativeLayout
... >
<EditText
    android:id="@+id/editText1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentLeft="true"
    android:layout_alignParentTop="true"
    android:ems="10" />
<Button
    android:id="@+id/button1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignBottom="@id/editText1"
    android:layout_toRightOf="@id/editText1"
    android:text="Un" />
<EditText
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/editText1"
    android:ems="10" />
</RelativeLayout>
```



- organisation des composants en liste (scrollable)
- propriétés :
 - `android:divider`, `android:dividerHeight` : aspect du séparateur d'item de liste
 - `android:entries` : tableau pour peupler la liste
- en fait nécessite un `Adapter`
- mise en page personnalisée ou standard

Example ListView



```
<ListView
    android:id="@+id/listView1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentLeft="true"
    android:layout_alignParentTop="true"
    android:layout_marginLeft="75dp"
    android:layout_marginTop="54dp"
    android:entries="@array/lorem" >
</ListView>
```

```
<resources>
    <string-array name="lorem">
        <item>lorem</item>
        <item>ipsum</item>
        <item>dolor</item>
        <item>sit</item>
        <item>amet</item>
        ...
        <item>leo</item>
    </string-array>
</resources>
```